

Séance 2 : Transformations temporelles et Silver Stream

Owner	 louis reynouard
Tags	

Contexte

Votre table `bronze_stream_taxi` grossit en temps réel. Les données sont brutes, non typées pour certaines colonnes, sans enrichissement géographique. Cette séance reproduit la logique Silver de Spark Core en flux continu, et ajoute deux notions absentes du batch : la gestion du temps et les watermarks.

1. Les trois visions du temps dans Spark Streaming

En streaming, "le moment d'un événement" peut vouloir dire trois choses différentes, et confondre les trois est une source d'erreurs classique.

Event Time : le moment où l'événement s'est produit dans le monde réel. Pour un trajet taxi : quand le client a pris le taxi (`tpep_pickup_datetime`). C'est la mesure métier correcte.

Ingestion Time : le moment où Spark a reçu la donnée. Correspond à votre colonne `ingestion_timestamp` . Utile pour le monitoring technique, mais trompeur pour les calculs métier si des données arrivent en retard.

Processing Time : le moment où Spark traite la donnée dans le micro-batch courant. Le plus simple à utiliser, le moins fiable pour l'analyse.

Règle pratique : pour les agrégations métier, utilisez l'**Event Time**. Pour le monitoring technique ("mon pipeline prend-il du retard ?"), comparez l'Event Time à l'Ingestion Time.

2. Les fenêtres temporelles

Spark Structured Streaming supporte trois types de fenêtres, toutes basées sur l'Event Time.

Fenêtre fixe (Tumbling Window)

Des tranches de temps qui ne se chevauchent pas. Un trajet à 14h07 tombe dans la fenêtre [14h00, 14h10). Un trajet à 14h11 tombe dans [14h10, 14h20). La fonction `window(colonne_event_time, "10 minutes")` produit une colonne structurée avec un champ `start` et un champ `end`. Elle s'utilise dans un `groupBy` comme n'importe quelle colonne.

Fenêtre glissante (Sliding Window)

Les fenêtres se chevauchent. Un trajet peut appartenir à plusieurs fenêtres simultanément. La fonction prend un deuxième argument : le pas d'avancement. `window(col, "10 minutes", "5 minutes")` crée des fenêtres de 10 minutes qui avancent toutes les 5 minutes. Chaque trajet appartient à deux fenêtres simultanément. Plus coûteux en mémoire car Spark maintient l'état de plusieurs fenêtres actives en parallèle.

Fenêtre de session

Une fenêtre qui reste ouverte tant que l'activité continue, et se ferme après un gap d'inactivité. Moins courante, pas abordée dans ce cours.

3. Le problème des données tardives

Dans la vie réelle, les données n'arrivent pas dans l'ordre. Un trajet terminé à 14h05 peut n'être envoyé au système qu'à 14h23, parce que le réseau du taxi avait un problème.

Sans gestion du retard, deux problèmes se posent :

Problème 1 : résultats incorrects. La fenêtre [14h00, 14h10] a été calculée à 14h10. Le trajet de 14h05 arrive à 14h23. Il n'est pas pris en compte. Votre métrique sous-estime le trafic réel.

Problème 2 : état infini. Spark ne sait pas quand il peut libérer la mémoire allouée à une fenêtre. Il garde tout en mémoire indéfiniment. Sur un compte gratuit avec 15 Go de RAM, le job crash en quelques heures.

4. Watermarks : la solution

Un watermark dit à Spark : "Je tolère des données en retard de X. Au-delà, je les ignore et je libère la mémoire."

```
df_with_watermark = (  
    df_stream  
    .withWatermark("tpep_pickup_datetime", "10 minutes")  
)
```

Avec ce watermark, si la donnée la plus récente que Spark a vue a un Event Time de 14h30, alors Spark considère que toute donnée avec un Event Time inférieur à 14h20 est "trop tardive" et peut être ignorée. Les fenêtres antérieures à 14h20 sont fermées et la mémoire est libérée.

Choisir la bonne valeur de watermark

C'est un compromis entre deux exigences contradictoires :

- **Watermark trop court** (ex : 30 secondes) : vous perdez des données légitimement en retard. Vos agrégations sont inexactes.
- **Watermark trop long** (ex : 2 heures) : Spark garde des fenêtres ouvertes très longtemps. Consommation mémoire élevée. Inacceptable sur compte gratuit.

Pour le TP, nous utiliserons des watermarks de 2 à 5 minutes, cohérents avec le simulateur. Le simulateur injecte par défaut un retard entre 1 et 10 minutes sur les lignes tardives. Si vous souhaitez contrôler précisément la proportion de données perdues (par exemple pour tester un watermark de 3 minutes), vous pouvez ajuster `late_data_min_minutes` et `late_data_max_minutes` dans la configuration du simulateur. La documentation du simulateur détaille cette mécanique.

5. Stream-Static Join : enrichir le flux

Vous avez déjà réalisé cette jointure en batch dans Spark Core S2, sur le même principe : joindre `PULocationID` sur `LocationID` pour obtenir le borough de départ. En streaming, la mécanique est identique à une différence près : un DataFrame est un flux (`readStream`), l'autre est statique (`read`).

Spark gère ce cas nativement : la table statique est chargée une fois en mémoire et utilisée pour enrichir chaque micro-batch.

Dans ce cours, la table statique des zones est `ref_taxi_zones`, construite à partir du fichier `taxi_zones.csv` fourni. Si vous avez également une table `silver_zones` fonctionnelle dans votre catalogue depuis Spark Core, vous pouvez l'utiliser à la place à condition qu'elle contienne les colonnes `LocationID` et `Borough`. La source importe peu ; ce qui compte, c'est que la colonne `pickup_borough` soit présente dans `silver_stream_taxi`.

La table des zones compte 265 lignes. Plutôt que de shuffler les données du flux pour les faire correspondre à ces 265 entrées, Spark envoie une copie complète de la petite table à chaque executor via `broadcast`. Chaque executor fait la jointure localement, sans échange réseau. C'est le même mécanisme que dans Spark Core S2, appliqué ici à un flux.



Quand Spark fait une jointure entre deux DataFrames, il doit amener les données des deux côtés au même endroit pour pouvoir les comparer. Si les deux tables sont grandes, il fait un **shuffle** : il redistribue les données sur le réseau entre les executors selon la clé de jointure. C'est une opération coûteuse, surtout en streaming où elle se répète à chaque micro-batch.

TP Séance 2 : Transformations temporelles et couche Silver streaming

Ce qui vous est donné

Le simulateur configuré avec `late_data_ratio: 0.15` (15% de trajets avec un Event Time en retard de 1 à 10 minutes selon la distribution interne du simulateur). C'est le seul paramètre imposé. Le reste de la configuration est votre décision, à justifier.

Ce que vous devez construire

Étape préalable : intégrer le fichier de référence des zones.

Le fichier `taxi_zones.csv` est fourni dans le repo du cours (265 lignes, colonnes `LocationID`, `Borough`, `Zone`, `service_zone`). Déposez-le dans votre Volume et intégrez-le dans votre catalogue sous forme de table Delta exploitable. C'est votre premier livrable de cette séance : la table doit exister et être interrogeable avant de lancer les jobs streaming.

Job 1 : Silver streaming.

À partir de la table `bronze_stream_taxi`, produire une table `silver_stream_taxi` avec les colonnes correctement typées (les mêmes que votre Silver Spark Core) et enrichie par la table des zones via un Stream-Static Join. Cette jointure se fait sur `PULocationID = LocationID` depuis la table `ref_taxi_zones` que vous venez de charger, pour obtenir la colonne `pickup_borough`.

Un watermark doit être appliqué sur `tpep_pickup_datetime`. Sa valeur est votre décision : justifiez-la en fonction du `late_data_ratio` du simulateur et des contraintes de mémoire du compte gratuit.

Si vous souhaitez contrôler précisément quelle proportion de données tardives est hors watermark, ajustez `late_data_min_minutes` et `late_data_max_minutes` dans le simulateur avant de lancer.

Job 2 : KPI par fenêtre.

À partir de `silver_stream_taxi`, produire une table `kpi_stream_by_window` contenant le nombre de trajets et le montant moyen par borough et par fenêtre de 10 minutes.

Ce job est séparé du premier : deux notebooks ou deux cellules distinctes, deux checkpoints distincts. Le mode de sortie est votre décision : justifiez-le.

Validation des données tardives.

Pendant la simulation, comparez le nombre de lignes dans `bronze_stream_taxi` et dans `silver_stream_taxi` à deux moments différents.

Consignez les deux relevés dans le notebook et commentez l'évolution du ratio.

Contraintes techniques

Deux jobs, deux checkpoints dans des chemins distincts sous votre Volume.
Aucune valeur de trigger sans justification dans une cellule Markdown. Les noms de tables et de chemins contiennent votre trigramme.

Livraison : Notebook **S2_Silver_Stream_TRIGRAMME**

Les deux jobs fonctionnels avec leurs justifications. La validation Bronze/Silver exécutée deux fois avec les deux sorties dans le notebook. Un `display()` de `kpi_stream_by_window` après au moins 15 minutes de simulation montrant plusieurs fenêtres pour au moins deux boroughs.

Deux cellules Markdown répondant aux questions suivantes:

Question 1. Vous avez `late_data_ratio=0.15` et le watermark que vous avez choisi. Que se passe-t-il si vous le divisez par 5 ? Quantifiez l'impact attendu sur le taux de données perdues. Montrez le calcul.

Question 2. Pourquoi le mode de sortie `complete` serait-il dangereux pour `kpi_stream_by_window` sur un compte Databricks gratuit ? Estimez à partir de vos mesures combien de mémoire serait nécessaire après 24h de simulation.

Grille d'évaluation

Critère	Insuffisant	Attendu	Excellent
Watermark	Absent ou valeur arbitraire	Valeur justifiée par rapport au simulateur	Valeur + mesure de l'impact observé
Mode de sortie	<code>append</code> pour tout	<code>update</code> pour les agrégations	• explication de pourquoi <code>complete</code> est exclu
Validation Bronze/Silver	Non réalisée	Ratio calculé une fois	Ratio calculé deux fois + évolution analysée
KPI par fenêtre	Absent	Table peuplée avec plusieurs fenêtres	• comparaison par borough avec commentaire

Critère	Insuffisant	Attendu	Excellent
Questions rédigées	Paraphrase	Réponse technique avec calcul	Calcul + confrontation aux observations réelles

Les réponses aux questions théoriques comptent pour 40% de la note de ce TP.